

Mini serveur web local puis exposé sur Internet

* * * * *

1 Objectif

1. Analyser le « build » du siteweb Astro, localement
2. Exposer temporairement ce « localhost » sur internet à des fins de démo-correction par un tiers

1.1 Étapes

1.1.a Build du website

Pages web « statiques » dans le dossier docs/ (config pour être servi par **Github** en y associant le nom de domaine « regrain.fr » au travers d'un document « CNAME ») :

```
npm run build
```

JS JavaScript

1.1.b Serveur «local»

1. **Option 1** : « simple » HTTP server with Python :

```
python -m http.server 8080 --directory docs
```

Python

1. **Option 2 (préférée)** : npx serve (répertoire docs/ sur port 3000)

```
npx serve docs/
```

JS JavaScript

Le serveur démarre sur le port **3000** et affiche :

```
Serving!
```

```
- Local:    http://localhost:3000  
- Network:  http://192.168.56.1:3000
```

```
Copied local address to clipboard!
```

1.1.c Exposer le localhost sur internet

Utiliser tunnl.gg : service de tunnel SSH, sans installation, gratuit.

```
ssh -t -R 80:localhost:3000 proxy.tunnl.gg
```

Shell

1.1.d Notes

- Aucune installation requise (sauf SSH à installer sur Windows 11 avec «winget install Microsoft.OpenSSH.Preview»)

- L'URL change à chaque nouvelle connexion
- Le tunnel se ferme automatiquement à la déconnexion SSH

2 Synthèse des commandes

1. **Build + lancer le serveur local** (dans un terminal) :

```
npm run build && npx serve docs/
```

JavaScript

2. **Ouvrir le tunnel** (dans un second terminal) :

```
ssh -t -R 80:localhost:3000 proxy.tunnel.gg
```

Shell

tunnel.gg affiche une URL publique (ex. <https://xxxx.tunnel.gg>) à partager avec le client.

3. **Fermer le tunnel** : Ctrl+C dans le terminal SSH.